IES Horizonte

Manuel Soltero Díaz
CFGS Administración de Sistemas Informáticos en Red
2025/2026
IES Albarregas
3/02/2026

Manual de programador de la aplicación de incidencias

Introducción..... 3
Vista Docente..... 4
Vista Administrador/Informático..... 8
Vista Director.....14
Conclusión..... 20

Introducción

En este manual se pretende explicar de forma clara y detallada el funcionamiento interno de la aplicación de gestión de incidencias del IES Horizonte, describiendo el código de cada una de las tres vistas que la componen: Vista Docente, Vista Administrador/Informático y Vista Director. Va dirigido a perfiles técnicos que necesiten entender, mantener o modificar la aplicación. Cabe destacar que la aplicación no dispone de entorno gráfico, por lo que se ejecuta directamente desde la terminal, y está diseñada para funcionar en sistemas operativos Windows prioritariamente.

La aplicación trabaja con un fichero llamado incidencias.txt que hay crearlo manualmente.

Esta aplicación tiene tres vistas claramente diferenciadas:

- Vista de Docente: donde los docentes del centro pueden crear incidencias sobre algún error, algo que no funciona, etc.
- Vista de Director: donde el director del centro pueda ver todas las incidencias y pueda filtrar por fecha o por un día en concreto.
- Vista de Informático: donde el informático del centro podrá ver las incidencias creadas por los docentes y podrá ir resolviendo dichas incidencias y se irán borrando del sistema cuando estén resueltas.

A continuación se procede a explicar el código de la aplicación de cada vista, al final del documento se compartirá una carpeta de Google Drive con los tres archivos correspondientes.

Vista Docente

1. Se debe importar dos librerías que son *os* para poder hacer llamadas al sistema operativo y *datetime* para poder usar la fecha actual del sistema.

```
import os
from datetime import datetime
```

2. A continuación se especifica el archivo donde se van a guardar las incidencias creadas.

```
# Ruta del archivo de incidencias en C:
ARCHIVO_INCIDENCIAS = "C:\IES Horizonte\incidencias.txt"
```

3. Aquí simplemente se crea una función para limpiar la pantalla de la terminal cada vez que se ejecute una instrucción, que se verá más adelante.

```
def limpiar pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')
```

4. Esta es la función principal de esta vista. Esta función es la encargada de crear la incidencia. Como se ve primero creamos la función *crear_incidencia* y sobre ella escribirnos el siguiente código. Llamamos a la función creada anteriormente *limpiar_pantalla* para que la pantalla de la terminal esté limpia y se vea mejor, después con un **print** se escribe el título (NUEVA INCIDENCIA). A continuación se empieza con el “formulario” para la creación de la incidencia. Se pide el título, una descripción, el nombre del docente lo “coge” el propio sistema operativo con la librería importada al principio con la variable de entorno **USERNAME** y escribe la app por pantalla el nombre de docente, seguidamente sale una lista con 3 opciones dependiendo de la categoría a la que vaya destinada la incidencia con una variable al final *cat_opcion* donde guarda el número de la categoría y debajo hay un array donde está guardado el número con un valor(palabra) donde se se elige la categoría con el número seleccionado anteriormente, si no elige ninguna se le asigna un valor por defecto, en este caso **Técnica.**

```
def crear_incidencia():
    limpiar_pantalla()
    print("-" * 50)
    print("  NUEVA INCIDENCIA")
    print("-" * 50)

    titulo = input("\nTítulo: ").strip()
    descripcion = input("Descripción detallada: ").strip()
    nombre_docente = os.environ["USERNAME"]
    print(nombre_docente)
```

```
print("\nCategoría:")
print("1. Técnica (ordenadores, proyectores, impresoras, red)")
print("2. Infraestructura (mobiliario, luces, climatización)")
print("3. Materiales (falta de recursos didácticos)")
cat_opcion = input("Seleccione categoría (1-3): ").strip()

categorias = {
    "1": "Técnica",
    "2": "Infraestructura",
    "3": "Materiales"
}
categoria = categorias.get(cat_opcion, "Técnica")
```

5. En esta parte se elige la ubicación desde donde está el error/fallo que lo deberá escribir el usuario que esté rellenando la incidencia. Seguidamente de nuevo hay otras tres opciones que funcionan igual que las categorías mencionadas anteriormente con tres prioridades, si no elige ninguna se le asigna la prioridad **Media.**

```
ubicacion = input("Ubicación (aula, laboratorio, etc.): ").strip()

print("\nPrioridad:")
print("1. Baja")
print("2. Media")
print("3. Alta")
prio_opcion = input("Seleccione prioridad (1-3): ").strip()

prioridades = {
    "1": "Baja",
    "2": "Media",
    "3": "Alta"
}
prioridad = prioridades.get(prio_opcion, "Media")
```

6. En este punto el programa usando la librería *datetime* guarda en una variable la fecha con este formato DIA/MES/AÑO y en otra variable la hora en que se crea la incidencia con el formato HORA/MINUTOS/SEGUNDOS,

```
fecha = datetime.now().strftime("%d/%m/%Y")
hora = datetime.now().strftime("%H:%M:%S")
estado = "Pendiente"
```

7. Ahora cuando la incidencia se ha escrito, se procede a escribir en el fichero el valor que han tomado las variables en la ruta que especificamos al principio del código, (**\n** es simplemente un salto de línea). Y al final del bloque hace un print para decir que la incidencia se ha registrado correctamente.

```
# Guardar en C:\IES Horizonte\incidencias.txt
with open(ARCHIVO_INCIDENCIAS, "a", encoding="utf-8") as archivo:
    archivo.write("-" * 50 + "\n")
    archivo.write(f"TÍTULO: {titulo}\n")
    archivo.write(f"DESCRIPCIÓN: {descripcion}\n")
    archivo.write(f"DOCENTE: {nombre_docente}\n")
    archivo.write(f"CATEGORÍA: {categoria}\n")
    archivo.write(f"UBICACIÓN: {ubicacion}\n")
    archivo.write(f"PRIORIDAD: {prioridad}\n")
    archivo.write(f"FECHA: {fecha}\n")
    archivo.write(f"HORA: {hora}\n")
    archivo.write(f"ESTADO: {estado}\n")
    archivo.write("-" * 50 + "\n\n")

print("\n Incidencia registrada correctamente")
input("\nPresione Enter para continuar...")
```

8. Al final del código definimos la función del bucle para poder elegir las funciones creadas anteriormente. Se ha utilizado un while true para que siempre la condición sea válida. La estructura es muy sencilla
   - Usamos la función *limpiar_pantalla* para borrar la pantalla.
   - Usamos **print** para poder escribir en la pantalla.
   - Definimos una variable llamada **opcion** donde guarda el valor que introduzca el usuario.
   - Se hace uso de una estructura condicional de in if,elif y else.
     - Si el usuario escribe **1** se mostrará la función de *crear_incidencia.*
     - Si el usuario escribe **2** el se limpiará la pantalla con la funcion *limpiar_pantalla* y el programa acabará y se cerrará.

- Si el usuario escribe algo que no sea ni **1** ni **2** el programa le volverá a pedir que introduzca una opción.

```
def main():
    while True:
        limpiar_pantalla()
        print("-" * 50)
        print("  SISTEMA DE REGISTRO DE INCIDENCIAS")
        print("  IES Horizonte - Vista Docente")
        print("-" * 50)
        print("\n1. Crear nueva incidencia")
        print("2. Salir")
        print("\n" + "-" * 50)

        opcion = input("\nSeleccione una opción: ").strip()

        if opcion == "1":
            crear_incidencia()
        elif opcion == "2":
            limpiar_pantalla()
            print("\n¡Hasta pronto!\n")
            break
        else:
            print("Opción no válida")
            input("\nPresione Enter para continuar...")
```

9. Por último llamamos a la función main (es la función que contiene el menú). Indica que el archivo se está ejecutando directamente como programa. Ejecuta la función principal del script en ese momento, cuando arranque el programa.

```
if __name__ == "__main__":
    main()
```

Vista Administrador/Informático

1. Importamos la librería **os** para hacer llamadas al sistema operativo.
   Se define la ruta donde están las incidencias para poder leer ese archivo.

```
import os

# Ruta del archivo de incidencias en C:
ARCHIVO_INCIDENCIAS = "C:\IES Horizonte\incidencias.txt"
```

2. Se define una función para poder limpiar la pantalla para que se vea lo que interesa.

```
def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')
```

3. Ahora se define la función *leer_incidencias* para poder acceder al archivo. Usamos un condicional **if** para saber si el archivo contiene información, si no existe el archivo o no hay incidencias salta un error que dice que no hay incidencias registradas.

   Después se crean dos listas vacías llamadas incidencias e incidencias_actual.

   El código **lee un archivo**, detecta cuándo empieza una nueva incidencia y va guardando cada campo (como “Fecha: …”, “Usuario: …”, etc.) en un diccionario que es la función *incidencia_actual*. Cada incidencia completa se añade a una lista que es incidencias. Y la función retorna lo que hay en la variable *incidencias*.

```
def leer_incidencias():
# Lee todas las incidencias del archivo y las devuelve como lista de
diccionarios
    if not os.path.exists(ARCHIVO_INCIDENCIAS):
        return []

    incidencias = []
    incidencia_actual = {}

    with open(ARCHIVO_INCIDENCIAS, "r", encoding="utf-8") as archivo:
        for linea in archivo:
            linea = linea.strip()
            if linea.startswith("-" * 50):
                if incidencia_actual:
                    incidencias.append(incidencia_actual)
```

```
                    incidencia_actual = {}
            elif ": " in linea and not linea.startswith("-"):
                clave, valor = linea.split(": ", 1)
                incidencia_actual[clave] = valor

    return incidencias
```

4. Aquí tenemos la variable *guardar_incidencias.*
   Abre el archivo incidencias.txt en modo escritura(w) que lo que hacer es borrar el contenido anterior y escribe desde cero y si no existe el archivo lo crea.
   Con el for recorre todas las incidencias y por cada incidencia hacer un ("-" * 50 + "\n") que significa que pone una línea de separación.
   Las demás líneas lo que hacen es escribir en el archivo y escribe otra línea de separación.
   En resumen sobrescribe el archivo y guarda todas las incidencias una por una, formateadas con líneas separadoras y cada campo en su propia línea.

   Es la función que actualiza el archivo cada vez que cambias un estado o eliminas una incidencia.

```
def guardar_incidencias(incidencias):
# Guarda las incidencias pendientes en el archivo
    with open(ARCHIVO_INCIDENCIAS, "w", encoding="utf-8") as archivo:
        for inc in incidencias:
            archivo.write("-" * 50 + "\n")
            archivo.write(f"TÍTULO: {inc.get('TÍTULO', '')}\n")
            archivo.write(f"DESCRIPCIÓN: {inc.get('DESCRIPCIÓN',
'')}\n")
            archivo.write(f"DOCENTE: {inc.get('DOCENTE', '')}\n")
            archivo.write(f"CATEGORÍA: {inc.get('CATEGORÍA', '')}\n")
            archivo.write(f"UBICACIÓN: {inc.get('UBICACIÓN', '')}\n")
            archivo.write(f"PRIORIDAD: {inc.get('PRIORIDAD', '')}\n")
            archivo.write(f"FECHA: {inc.get('FECHA', '')}\n")
            archivo.write(f"HORA: {inc.get('HORA', '')}\n")
            archivo.write(f"ESTADO: {inc.get('ESTADO', '')}\n")
            archivo.write("-" * 50 + "\n\n")
```

5. Esta función muestra las incidencias que hay en el archivo.
   Empieza con un **if**, lo que hacer es que sino la variable incidencias está vacía retorna false y acaba.

Después con un for vamos cogiendo los campos de la variable incidencias que es una lista y va mostrando en cada línea la incidencia en su campo correspondiente.

```
def mostrar_incidencias(incidencias):
# Muestra todas las incidencias numeradas
    if not incidencias:
        print("No hay incidencias registradas")
        return False

    for i, inc in enumerate(incidencias, 1):
        print(f"\n{'-' * 50}")
        print(f"  INCIDENCIA #{i}")
        print(f"{'-' * 50}")
        print(f"TÍTULO: {inc.get('TÍTULO', '')}")
        print(f"DESCRIPCIÓN: {inc.get('DESCRIPCIÓN', '')}")
        print(f"DOCENTE: {inc.get('DOCENTE', '')}")
        print(f"CATEGORÍA: {inc.get('CATEGORÍA', '')}")
        print(f"UBICACIÓN: {inc.get('UBICACIÓN', '')}")
        print(f"PRIORIDAD: {inc.get('PRIORIDAD', '')}")
        print(f"FECHA: {inc.get('FECHA', '')}")
        print(f"HORA: {inc.get('HORA', '')}")
        print(f"ESTADO: {inc.get('ESTADO', '')}")

    return True
```

```
        print("Debe introducir un número")
        input("\nPresione Enter para continuar...")
        return

    incidencia = incidencias[num - 1]

    print(f"\nIncidencia seleccionada: {incidencia.get('TÍTULO', '')}")
    print(f"Estado actual: {incidencia.get('ESTADO', '')}")

    print("\nNuevo estado:")
    print("1. Pendiente")
    print("2. Resuelto")

    opcion = input("\nSeleccione nuevo estado (1-2): ").strip()

    if opcion == "1":
        incidencias[num - 1]['ESTADO'] = "Pendiente"
```

```
        guardar_incidencias(incidencias)
        print("\n Estado cambiado a Pendiente")
    elif opcion == "2":
        incidencias[num - 1]['ESTADO'] = "Resuelto"
        # Eliminar la incidencia resuelta
        incidencias.pop(num - 1)
        guardar_incidencias(incidencias)
        print("\n Incidencia marcada como Resuelta y eliminada del
sistema")
    else:
        print("Opción no válida")

    input("\nPresione Enter para continuar...")
```

6. Función cambiar_estado
    - La función permite al informático cambiar el estado de una incidencia.
    - Primero limpia la pantalla y muestra todas las incidencias guardadas.
    - Luego el informático elige cuál quiere modificar escribiendo su número.
    - El programa verifica que el número sea válido y muestra el estado actual.
    - Después el informático elige si quiere ponerla como Pendiente o Resuelta.
    - Si se marca como Pendiente, solo se actualiza el estado.
    - Si se marca como Resuelta, además de cambiar el estado, la incidencia se elimina del listado.
    - Finalmente, se guardan los cambios en el archivo .

```
def cambiar_estado():
# Permite cambiar el estado de una incidencia
    limpiar_pantalla()
    print("-" * 50)
    print("  CAMBIAR ESTADO DE INCIDENCIA")
    print("-" * 50)

    incidencias = leer_incidencias()

    if not mostrar_incidencias(incidencias):
        input("\nPresione Enter para continuar...")
        return

    print("\n" + "-" * 50)
    num = input("\n¿Qué incidencia desea cambiar? (número): ").strip()

    try:
```

```
        num = int(num)
        if num < 1 or num > len(incidencias):
            print("Número de incidencia no válido")
            input("\nPresione Enter para continuar...")
            return
    except ValueError:
        print("Debe introducir un número")
        input("\nPresione Enter para continuar...")
        return

    incidencia = incidencias[num - 1]

    print(f"\nIncidencia seleccionada: {incidencia.get('TÍTULO', '')}")
    print(f"Estado actual: {incidencia.get('ESTADO', '')}")

    print("\nNuevo estado:")
    print("1. Pendiente")
    print("2. Resuelto")

    opcion = input("\nSeleccione nuevo estado (1-2): ").strip()

    if opcion == "1":
        incidencias[num - 1]['ESTADO'] = "Pendiente"
        guardar_incidencias(incidencias)
        print("\n Estado cambiado a Pendiente")
    elif opcion == "2":
        incidencias[num - 1]['ESTADO'] = "Resuelto"
        # Eliminar la incidencia resuelta
        incidencias.pop(num - 1)
        guardar_incidencias(incidencias)
        print("\n Incidencia marcada como Resuelta y eliminada del
sistema")
    else:
        print("Opción no válida")

    input("\nPresione Enter para continuar...")
```

7. La función limpia la pantalla y muestra un encabezado con el título del sistema. Luego lee todas las incidencias guardadas en el archivo y las muestra en pantalla usando la función *ver_incidencias().*
   Finalmente imprime una línea de cierre para separar visualmente el listado.

```
def ver_incidencias():
# Muestra todas las incidencias
```

```
    limpiar_pantalla()
    print("-" * 50)
    print("  LISTADO DE INCIDENCIAS")
    print("  IES Horizonte - Vista Informático")
    print("-" * 50)

    incidencias = leer_incidencias()
    mostrar_incidencias(incidencias)

    print("\n" + "-" * 50)
```

8. Al final del código definimos la función del bucle para poder elegir las funciones creadas anteriormente. Se ha utilizado un while true para que siempre la condición sea válida. La estructura es muy sencilla
    - Usamos la función *limpiar_pantalla* para borrar la pantalla.
    - Usamos **print** para poder escribir en la pantalla.
    - Definimos una variable llamada **opcion** donde guarda el valor que introduzca el usuario.
    - Se hace uso de una estructura condicional de in if,elif y else.
        - Si el informático escribe **1** se mostrará la función de *ver_incidencia.*
        - Si el informático escribe **2** puede cambiar el estado de la incidencia
        - Si el informático escribe **3** sale de la aplicación.
        - Si el informatico escribe algo que no sea ni **1** ni **2** ni **3** el programa le volverá a pedir que introduzca una opción.

```
def main():
    while True:
        limpiar_pantalla()
        print("-" * 50)
        print("  SISTEMA DE GESTIÓN DE INCIDENCIAS")
        print("  IES Horizonte - Vista Informático")
        print("-" * 50)
        print("\n1. Ver todas las incidencias")
        print("2. Cambiar estado de incidencia")
        print("3. Salir")
        print("\n" + "-" * 50)

        opcion = input("\nSeleccione una opción: ").strip()

        if opcion == "1":
            ver_incidencias()
```

```
            input("\nPresione Enter para continuar...")
        elif opcion == "2":
            cambiar_estado()
        elif opcion == "3":
            limpiar_pantalla()
            print("\n¡Hasta pronto!\n")
            break
        else:
            print("Opción no válida")
            input("\nPresione Enter para continuar...")
```

9. Por último llamamos a la función main (es la función que contiene el menú). Indica que el archivo se está ejecutando directamente como programa. Ejecuta la función principal del script en ese momento, cuando arranque el programa.

```
if __name__ == "__main__":
    main()
```

Vista Director

1. Importamos la librería **os** para hacer llamadas al sistema operativo. Se define la ruta donde están las incidencias para poder leer ese archivo.

```
import os

# Ruta del archivo de incidencias en C:
ARCHIVO_INCIDENCIAS = "C:\IES Horizonte\incidencias.txt"
```

2. Se define una función para poder limpiar la pantalla para que se vea lo que interesa.

```
def limpiar_pantalla():
    os.system('cls' if os.name == 'nt' else 'clear')
```

3. Ahora se define la función *leer_incidencias* para poder acceder al archivo. Usamos un condicional **if** para saber si el archivo contiene información, si no existe el archivo o no hay incidencias salta un error que dice que no hay incidencias registradas.

   Después se crean dos listas vacías llamadas incidencias e incidencias_actual.

El código **lee un archivo**, detecta cuándo empieza una nueva incidencia y va guardando cada campo (como “Fecha: …”, “Usuario: …”, etc.) en un diccionario que es la función *incidencia_actual*. Cada incidencia completa se añade a una lista que es incidencias. Y la función retorna lo que hay en la variable *incidencias*.

```
def leer_incidencias():
# Lee todas las incidencias del archivo y las devuelve como lista de
diccionarios
    if not os.path.exists(ARCHIVO_INCIDENCIAS):
        return []

    incidencias = []
    incidencia_actual = {}

    with open(ARCHIVO_INCIDENCIAS, "r", encoding="utf-8") as archivo:
        for linea in archivo:
            linea = linea.strip()
            if linea.startswith("-" * 50):
                if incidencia_actual:
                    incidencias.append(incidencia_actual)
                    incidencia_actual = {}
            elif ": " in linea and not linea.startswith("-"):
                clave, valor = linea.split(": ", 1)
                incidencia_actual[clave] = valor

    return incidencias
```

4. La función muestra un listado ordenado de incidencias en pantalla.
   Presenta un encabezado con el título y, si no hay incidencias, informa que la lista está vacía.
   Si existen, recorre cada incidencia y muestra sus datos (título, descripción, docente, categoría, ubicación, prioridad, fecha, hora y estado), numerándolas una por una.
   Al final imprime el total de incidencias registradas.

```
def mostrar_incidencias(incidencias, titulo="LISTADO DE INCIDENCIAS"):
# Muestra todas las incidencias
    limpiar_pantalla()
    print("-" * 50)
    print(f"  {titulo}")
    print("  IES Horizonte - Vista Director")
    print("-" * 50)

    if not incidencias:
```

```
        print("\nNo hay incidencias registradas")
        return

    for i, inc in enumerate(incidencias, 1):
        print(f"\n{'-' * 50}")
        print(f"  INCIDENCIA #{i}")
        print(f"{'-' * 50}")
        print(f"TÍTULO: {inc.get('TÍTULO', '')}")
        print(f"DESCRIPCIÓN: {inc.get('DESCRIPCIÓN', '')}")
        print(f"DOCENTE: {inc.get('DOCENTE', '')}")
        print(f"CATEGORÍA: {inc.get('CATEGORÍA', '')}")
        print(f"UBICACIÓN: {inc.get('UBICACIÓN', '')}")
        print(f"PRIORIDAD: {inc.get('PRIORIDAD', '')}")
        print(f"FECHA: {inc.get('FECHA', '')}")
        print(f"HORA: {inc.get('HORA', '')}")
        print(f"ESTADO: {inc.get('ESTADO', '')}")

    print(f"\n{'-' * 50}")
    print(f"Total de incidencias: {len(incidencias)}")
    print("-" * 50)
```

5. La función carga todas las incidencias guardadas, las muestra sin aplicar ningún filtro y luego espera a que el usuario pulse Enter para continuar.

   Primero obtiene la lista completa mediante *leer_incidencias*. Después llama a *mostrar_incidencias* para imprimirlas con el título “TODAS LAS INCIDENCIAS”. Finalmente detiene la ejecución hasta que el usuario presione Enter.

```
def ver_todas():
# Muestra todas las incidencias sin filtro
    incidencias = leer_incidencias()
    mostrar_incidencias(incidencias, "TODAS LAS INCIDENCIAS")
    input("\nPresione Enter para continuar...")
```

6. La función permite mostrar solo las incidencias que coinciden con una fecha concreta introducida por el usuario.

   Primero carga todas las incidencias. Si no hay ninguna, limpia la pantalla, informa de que no existen registros y espera a que el usuario pulse Enter.
   Sí hay incidencias, muestra un encabezado y pide una fecha en formato DD/MM/YYYY.
   Después crea una lista nueva con solo las incidencias cuya clave 'FECHA' coincide exactamente con la fecha introducida.

Si encuentra resultados, los muestra usando *mostrar_incidencias* con un título personalizado.
Si no encuentra ninguna, limpia la pantalla y avisa de que no hay incidencias para esa fecha.
Finalmente espera a que el usuario pulse Enter para continuar.

```
def filtrar_por_fecha():
# Filtra incidencias por fecha
    incidencias = leer_incidencias()

    if not incidencias:
        limpiar_pantalla()
        print("No hay incidencias registradas")
        input("\nPresione Enter para continuar...")
        return

    limpiar_pantalla()
    print("-" * 50)
    print("  FILTRAR POR FECHA")
    print("-" * 50)

    fecha = input("\nIntroduzca la fecha (DD/MM/YYYY): ").strip()

    incidencias_filtradas = [inc for inc in incidencias if
inc.get('FECHA', '') == fecha]

    if incidencias_filtradas:
        mostrar_incidencias(incidencias_filtradas, f"INCIDENCIAS DEL
{fecha}")
    else:
        limpiar_pantalla()
        print(f"\nNo hay incidencias registradas para la fecha
{fecha}")

    input("\nPresione Enter para continuar...")
```

7. La función permite mostrar solo las incidencias cuyo **día** coincide con el valor introducido por el usuario.

   Primero carga todas las incidencias. Si no existe ninguna, limpia la pantalla, avisa y espera a que el usuario pulse Enter.
   Si hay registros, muestra un encabezado y pide un día en formato **DD**.
   Después crea una lista con las incidencias cuya fecha empieza por ese día seguido de “/”, lo que permite filtrar sin importar mes o año.

Si encuentra coincidencias, las muestra con *mostrar_incidencias* usando un título personalizado.
Si no encuentra ninguna, limpia la pantalla y avisa de que no hay incidencias para ese día.
Al final espera a que el usuario pulse Enter para continuar.

```
def filtrar_por_dia():
# Filtra incidencias por día (DD)
    incidencias = leer_incidencias()

    if not incidencias:
        limpiar_pantalla()
        print("No hay incidencias registradas")
        input("\nPresione Enter para continuar...")
        return

    limpiar_pantalla()
    print("-" * 50)
    print("  FILTRAR POR DÍA")
    print("-" * 50)

    dia = input("\nIntroduzca el día (DD): ").strip()

    incidencias_filtradas = [inc for inc in incidencias if
inc.get('FECHA', '').startswith(dia + "/")]

    if incidencias_filtradas:
        mostrar_incidencias(incidencias_filtradas, f"INCIDENCIAS DEL
DÍA {dia}")
    else:
        limpiar_pantalla()
        print(f"\nNo hay incidencias registradas para el día {dia}")

    input("\nPresione Enter para continuar...")
```

8. Al final del código definimos la función del bucle para poder elegir las funciones creadas anteriormente. Se ha utilizado un while true para que siempre la condición sea válida. La estructura es muy sencilla
    - Usamos la función *limpiar_pantalla* para borrar la pantalla.
    - Usamos **print** para poder escribir en la pantalla.
    - Definimos una variable llamada **opcion** donde guarda el valor que introduzca el usuario.

- Se hace uso de una estructura condicional de in if,elif y else.
  - Si el director escribe **1** se mostrará la función de *ver_todas.*
  - Si el director escribe **2** puede filtrar por fecha.
  - Si el director escribe **3** puede filtrar por día.
  - Si el director escribe **4** se acaba el programa
  - Si el director escribe algo que no sea ni **1** ni **2** ni **3** ni **4** el programa le volverá a pedir que introduzca una opción.

```
def main():
    while True:
        limpiar_pantalla()
        print("-" * 50)
        print("  SISTEMA DE CONSULTA DE INCIDENCIAS")
        print("  IES Horizonte - Vista Director")
        print("-" * 50)
        print("\n1. Ver todas las incidencias")
        print("2. Filtrar por fecha completa (DD/MM/YYYY)")
        print("3. Filtrar por día (DD)")
        print("4. Salir")
        print("\n" + "-" * 50)

        opcion = input("\nSeleccione una opción: ").strip()

        if opcion == "1":
            ver_todas()
        elif opcion == "2":
            filtrar_por_fecha()
        elif opcion == "3":
            filtrar_por_dia()
        elif opcion == "4":
            limpiar_pantalla()
            print("\n¡Hasta pronto!\n")
            break
        else:
            print("Opción no válida")
            input("\nPresione Enter para continuar...")
```

9. Por último llamamos a la función main (es la función que contiene el menú). Indica que el archivo se está ejecutando directamente como programa. Ejecuta la función principal del script en ese momento, cuando arranque el programa.

```
if __name__ == "__main__":
    main()
```

Conclusión

En conclusión, la aplicación de gestión de incidencias del IES Horizonte es una solución funcional y ligera que permite gestionar los problemas técnicos del centro de forma organizada y sin necesidad de entorno gráfico. A lo largo de este manual se ha explicado el código de cada una de las tres vistas, detallando el funcionamiento de cada función y su propósito dentro del programa. Gracias a su estructura modular, la aplicación es fácilmente mantenible y ampliable en el futuro por cualquier técnico que haya leído este documento.